

Backpropagation from Scratch: A Beginner's Guide

A Deep Dive into the Micrograd Engine

1 The Core Concept: The “Paper Trail”

In standard programming, if you calculate $c = a + b$, the computer gives you the result and immediately forgets how it got there. In Deep Learning, we need the computer to remember.

Think of our `Value` object as a piece of paper that records:

1. **The Data:** The current number (e.g., 5.0).
2. **The Ancestry:** Which other pieces of paper were used to calculate this one?
3. **The Operation:** Was it addition? Multiplication?
4. **The Gradient:** How much does this specific number affect the final result?

1.1 The Chain Rule (The Secret Sauce)

The entire engine relies on the **Chain Rule**. If z depends on y , and y depends on x , then the influence of x on z is:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

In our code, `self.grad` represents $\frac{dL}{d(\text{self})}$, where L is the final output.

2 The Implementation: Line-by-Line

2.1 1. The Constructor

We initialize every number with a gradient of 0. We don't know the influence yet!

```
1 class Value:
2     def __init__(self, data, _children=(), _op='', _label=''):
3         self.data = data
4         self.grad = 0.0 # Initially, this number has no known impact
5         self._prev = set(_children) # The "parents"
6         self._backward = lambda: None # Empty "recipe" for math
```

2.2 2. Addition: Passing the Influence

When we add two numbers, $out = a + b$, the derivative of out with respect to a is just 1. So, a simply inherits whatever gradient out has.

```

1 def __add__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data + other.data, (self, other), '+')
4
5     def _backward():
6         self.grad += 1.0 * out.grad
7         other.grad += 1.0 * out.grad
8     out._backward = _backward
9     return out

```

Note: We use += because if a variable is used twice, its gradients should accumulate (add up).

2.3 3. Multiplication: The Switcheroo

If $out = a \cdot b$, the derivative with respect to a is b .

```

1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*')
4
5     def _backward():
6         self.grad += other.data * out.grad
7         other.grad += self.data * out.grad
8     out._backward = _backward
9     return out

```

3 The “Brain” of the Operation: Topological Sort

We cannot calculate a parent’s gradient until we have calculated the child’s gradient. To do this, we lay all our “pieces of paper” out in a line such that every parent comes **after** its children. Then, we walk through that line backwards.

4 Full Code Listing

```

1 import math
2
3 class Value:
4     def __init__(self, data, _children=(), _op='', label=''):
5         self.data = data
6         self.grad = 0.0
7         self._backward = lambda: None
8         self._prev = set(_children)
9         self._op = _op
10        self.label = label
11
12    def __add__(self, other):
13        other = other if isinstance(other, Value) else Value(other)
14        out = Value(self.data + other.data, (self, other), '+')
15        def _backward():
16            self.grad += 1.0 * out.grad
17            other.grad += 1.0 * out.grad
18        out._backward = _backward
19        return out
20
21    def __mul__(self, other):

```

```

22         other = other if isinstance(other, Value) else Value(other)
23         out = Value(self.data * other.data, (self, other), '*')
24         def _backward():
25             self.grad += other.data * out.grad
26             other.grad += self.data * out.grad
27         out._backward = _backward
28         return out
29
30     def tanh(self):
31         x = self.data
32         t = (math.exp(2*x) - 1)/(math.exp(2*x) + 1)
33         out = Value(t, (self, ), 'tanh')
34         def _backward():
35             self.grad += (1 - t**2) * out.grad
36         out._backward = _backward
37         return out
38
39     def backward(self):
40         topo = []
41         visited = set()
42         def build_topo(v):
43             if v not in visited:
44                 visited.add(v)
45                 for child in v._prev:
46                     build_topo(child)
47             topo.append(v)
48         build_topo(self)
49         self.grad = 1.0
50         for node in reversed(topo):
51             node._backward()

```

5 Step-by-Step Explanation of the Micrograd Notebook

5.1 1. Importing Libraries

```

1 import numpy as np
2 import math
3 import graphviz
4 import matplotlib.pyplot as plt

```

Explanation:

- `numpy (np)`: Helps us work with numbers and arrays easily.
- `math`: Lets us use mathematical functions like exponentials.
- `graphviz`: Used to draw diagrams showing how numbers are connected.
- `matplotlib.pyplot (plt)`: Used to make plots and graphs.

5.2 2. The Value Class: The Heart of Everything

```

1 class Value:
2     def __init__(self, data, _children=(), _op='', _label=''):
3         self.data = data
4         self._prev = set(_children)

```

```

5     self._op = _op
6     self._label = _label
7     self.grad = 0.0
8     self._backward = lambda:None

```

Explanation:

- `data`: The actual number we are working with.
- `_prev`: The numbers that were used to make this one (its parents).
- `_op`: What operation made this number? (e.g., +, *)
- `_label`: A name for this number, just for us to keep track.
- `grad`: How much this number affects the final result (used for learning).
- `_backward`: A recipe for how to send influence (gradients) backward.

5.3 3. Printing and Adding Numbers

```

1     def __repr__(self):
2         return f"Value(data={self.data})"
3
4     def __add__(self, other):
5         out = Value(self.data + other.data, (self, other), "+")
6         def _backward():
7             self.grad += 1.0*out.grad
8             other.grad += 1.0*out.grad
9         out._backward = _backward
10        return out

```

Explanation:

- `__repr__`: Lets us print the Value nicely.
- `__add__`: Lets us add two Value numbers together. It also sets up how to send influence backward for learning.

5.4 4. Multiplying Numbers

```

1     def __mul__(self, other):
2         out = Value(self.data * other.data, (self, other), '*)
3         def _backward():
4             self.grad += other.data*out.grad
5             other.grad += self.data*out.grad
6         out._backward = _backward
7         return out

```

Explanation:

- `__mul__`: Multiplies two numbers and sets up the backward recipe for learning.

5.5 5. Tanh Activation: Squashing the Number

```
1 def tanh(self):
2     n = self.data
3     t = ((math.exp(2*n)-1)/(math.exp(2*n)+1))
4     out=Value(t,(self, ),"tanh")
5     def _backward():
6         self.grad += (1-t**2)*out.grad
7     out._backward = _backward
8     return out
```

Explanation:

- **tanh**: Squashes the number between -1 and 1. This is useful for neural networks to keep numbers manageable.

$$1 - \tanh^2(x)$$

5.6 6. Backward Pass: Sending Influence Backwards

```
1 def backward(self):
2     topo = []
3     visited = set()
4     def build_topo(v):
5         if v not in visited:
6             visited.add(v)
7             for child in v._prev:
8                 build_topo(child)
9             topo.append(v)
10    build_topo(self)
11    self.grad = 1.0
12    for node in reversed(topo):
13        node._backward()
```

Explanation:

- This function figures out the order to send gradients backward so every number gets its correct influence.
- It starts from the final result and works backward through all the numbers.

5.7 7. Creating Input Values

```
1 a = Value(2.0, _label='a')
2 b = Value(-3.0, _label='b')
3 c = Value(10.0, _label='c')
4 f = Value(-2.0, _label='f')
```

Explanation:

- We create four numbers: a, b, c, and f. Each is wrapped in a Value object so we can track its history and influence.
- **_label** is just a name to help us keep track of which number is which.

5.8 8. Forward Pass: Calculating the Output

```
1 e = a * b
2 e._label = "e"
3 d = e + c
4 d._label = 'd'
5 L = d * f
6 L._label = 'L'
```

Explanation:

- We combine our numbers using multiplication and addition, just like you would in math class.
- Each step creates a new Value object, which remembers how it was made.
- L is our final output, the result of all these operations.

5.9 9. Visualizing the Computation Graph

```
1 from graphviz import Digraph
2
3 def trace(root):
4     nodes, edges = set(), set()
5     def build(v):
6         if v not in nodes:
7             nodes.add(v)
8             for child in v._prev:
9                 edges.add((child, v))
10                build(child)
11    build(root)
12    return nodes, edges
13
14 def draw_dot(root):
15     dot = Digraph(format='svg', graph_attr={"rankdir": "LR"})
16     nodes, edges = trace(root)
17     for n in nodes:
18         uid = str(id(n))
19         dot.node(name=uid, label="{%s | data %.4f | grad %.4f}" % (n._label, n.
20         data, n.grad), shape='record')
21         if n._op:
22             dot.node(name=uid + n._op, label=n._op)
23             dot.edge(uid + n._op, uid)
24     for n1, n2 in edges:
25         dot.edge(str(id(n1)), str(id(n2)) + n2._op)
26     return dot
```

Explanation:

- This code draws a picture showing how all our numbers are connected.
- Each box shows a number, its value, and its gradient.
- Arrows show which numbers were used to make other numbers.

5.10 10. Drawing the Graph

```
1 draw_dot(L)
```

Explanation: This draws the computation graph for our final output L, so we can see the "paper trail" of calculations.

5.11 11. Gradient Descent: Learning by Nudging

```
1 learning_rate = 0.01
2
3 a.data += learning_rate * a.grad
4 b.data += learning_rate * b.grad
5 c.data += learning_rate * c.grad
6 f.data += learning_rate * f.grad
7
8 # Recompute forward pass
9 e = a * b
10 d = e + c
11 L = d * f
12
13 print(L.data)
```

Explanation:

- `learning_rate`: A small number that controls how much we change our values to improve the result.
- We "nudge" each number in the direction that makes the output better (lower or higher, depending on our goal).
- After nudging, we recalculate everything to see the new result.
- `print(L.data)` shows us the new output after learning.

5.12 12. Manual Gradients: Setting Influence by Hand

```
1 L.grad = 1.0
2 d.grad = -2.0
3 f.grad = 4.0
4 e.grad = -2.0
5 c.grad = -2.0
6 a.grad = 6.0
7 b.grad = -4.0
```

Explanation:

- Here, we set the gradients (influence) for each number by hand, as if we calculated them ourselves.
- This is useful for understanding how each number affects the final result.

5.13 13. Plotting the Tanh Function

```
1 plt.plot(np.arange(-5,5,0.2),np.tanh(np.arange(-5,5,0.2)))
2 plt.grid()
```

Explanation:

- This draws a graph of the tanh function, showing how it squashes numbers between -1 and 1.
- The grid helps us see the shape of the curve.

5.14 14. Building a Simple Neuron

```
1 #inputs X1,X2
2 x1 = Value(2.0, _label="x1")
3 x2 = Value(0.0, _label="x2")
4 #weights
5 w1 = Value(-3.0, _label="w1")
6 w2 = Value(1.0, _label="w2")
7 #bias of the neuron
8 b = Value(6.8813735, _label="bias")
9 x1w1 = x1 * w1; x1w1._label = "x1*w1"
10 x2w2 = x2 * w2; x2w2._label = "x2*w2"
11 x1w1x2w2 = x1w1 + x2w2; x1w1x2w2._label = "x1*w1 + x2*w2"
12 n = x1w1x2w2 + b; n._label = "n"
13 o = n.tanh(); o._label = "o"
```

Explanation:

- **x1, x2:** These are the inputs to our neuron (like features in a dataset).
- **w1, w2:** These are weights, which tell us how important each input is.
- **b:** The bias, which shifts the output up or down.
- We multiply each input by its weight, add them together, and add the bias.
- **n:** The total input to the neuron before activation.
- **o:** The output after applying the tanh activation, squashing the result between -1 and 1.

5.15 15. Visualizing the Neuron

```
1 draw_dot(o)
```

Explanation: This draws the computation graph for the neuron's output, showing all the steps and connections.